Dynamic Programming

Name: Y.Thanushma

Registration Number: 192425355

Course Code: CSA0617-Design and Analysis of Algorithms

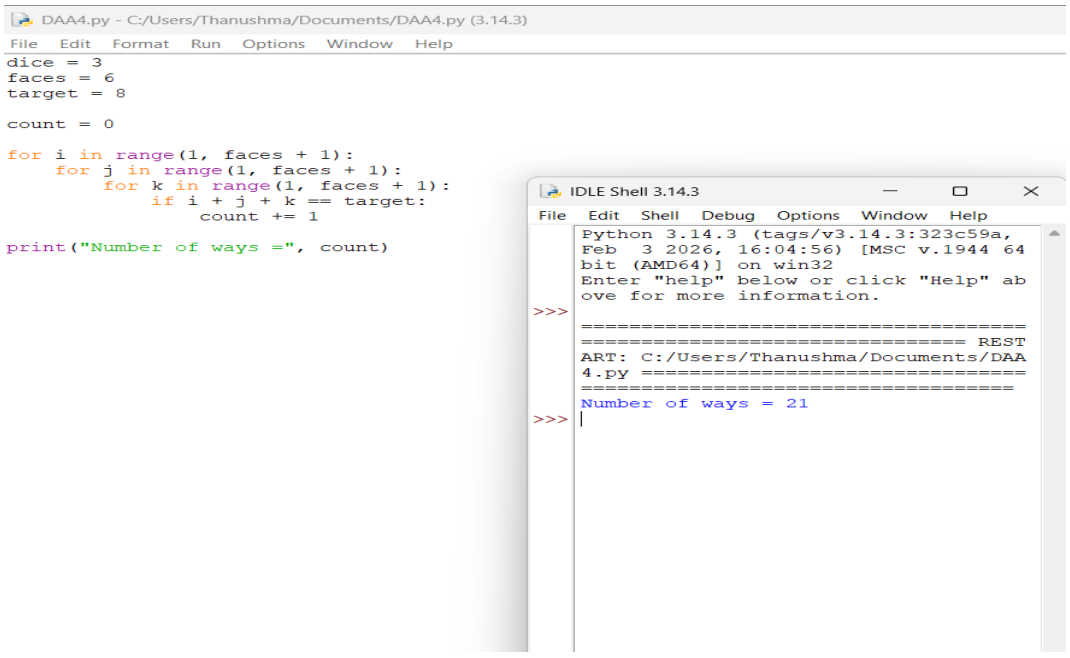
1. Dice Throw Problem

Aim: To write a Python program to determine the number of ways to obtain a target score of 8 using 3 six-faced dice.

Algorithm

1. Start.
2. Set the number of dice as 3, faces as 6, and target score as 8.
3. Initialize count = 0.
4. Generate all possible values of the three dice.
5. Check whether their sum is equal to 8.
6. If equal, increment count.
7. Display the total number of ways.
8. Stop.

Execution:



Result: Number of ways = 21

Thus, there are 21 possible combinations of 3 dice that produce a total score of 8**.**

2: Board Game Tournament

Aim: To write a Python program to calculate the number of possible outcomes when a player rolls 2 six-faced dice and the target score is 7.

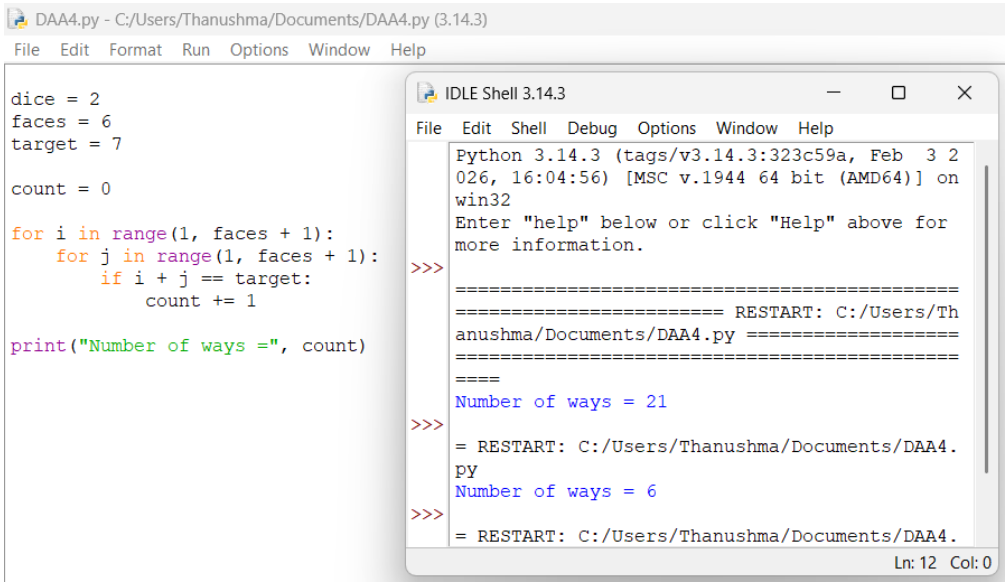
Input

- Number of Dice = 2
- Faces per Dice = 6
- Target Score = 7

Algorithm

1. Start.
2. Set the number of dice as 2, faces as 6, and target score as 7.
3. Initialize count = 0.
4. Generate all possible values for both dice.
5. Check if the sum of the two dice is equal to 7.
6. If the sum is 7, increment count.
7. Display the total number of ways.

Execution:



Result: The program successfully calculates that there are 6 possible outcomes to obtain a target score of 7 using two six-faced dice.

3: Warehouse Robot Navigation

Aim: To write a Python program to determine the number of possible movement sequences for a warehouse robot using 4 dice, each having 4 faces, where the total score must be 10.

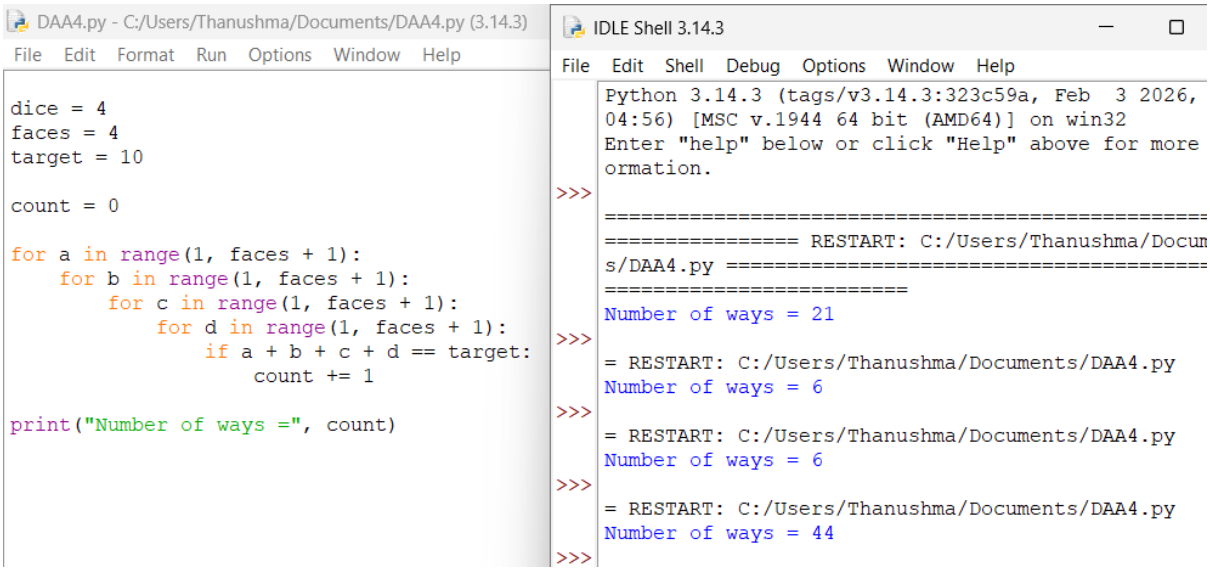
Input

- Number of Dice = 4
- Faces per Dice = 4
- Target Distance = 10

Algorithm

1. Start.
2. Set the number of dice as 4, faces as 4, and target distance as 10.
3. Initialize count = 0.
4. Generate all possible values for the 4 dice.
5. Check whether the sum of all four dice is equal to 10.
6. If the sum is 10, increment count.
7. Display the total number of possible sequences.

Execution:



Result: The program successfully determines that there are 44 possible movement sequences whose total distance is 10.

4.Casino Reward Calculation

Aim: To write a Python program to calculate the total number of winning combinations when 3 six-faced dice are rolled and the target sum is 12.

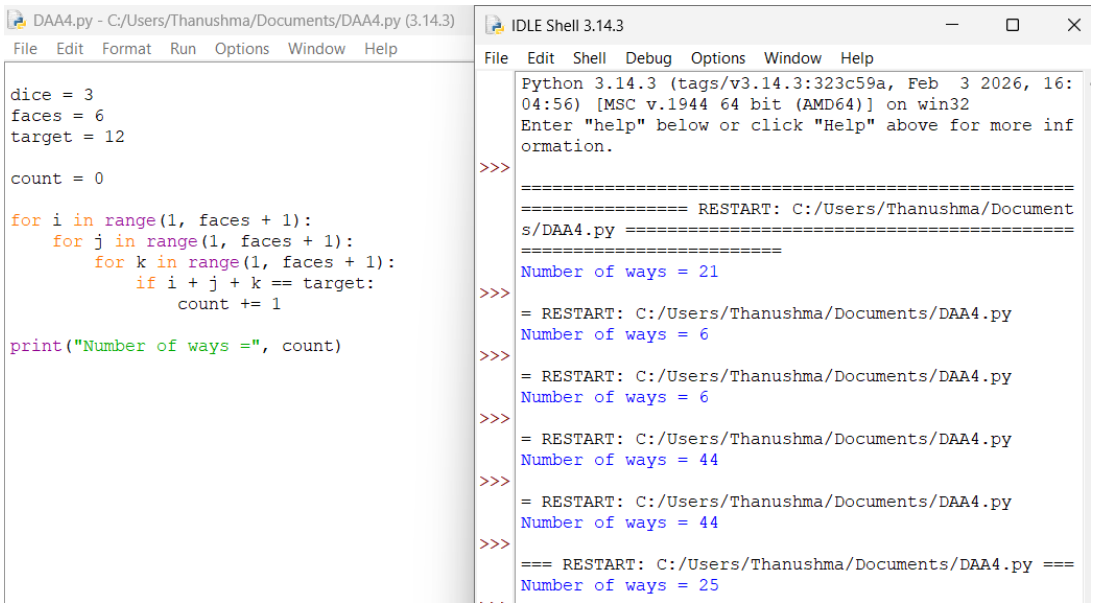
Input

- Number of Dice = 3
- Faces per Dice = 6
- Target Sum = 12

Algorithm

1. Start.
2. Set the number of dice as 3, faces as 6, and target sum as 12.
3. Initialize count = 0.
4. Generate all possible values for the three dice.
5. Check whether the sum of the three dice is equal to 12.
6. If the sum is 12, increment count.
7. Display the total number of winning combinations.
8. Stop.

Execution:



Result: The program successfully calculates that there are 25 winning combinations for obtaining a target sum of 12 using 3 six-faced dice.

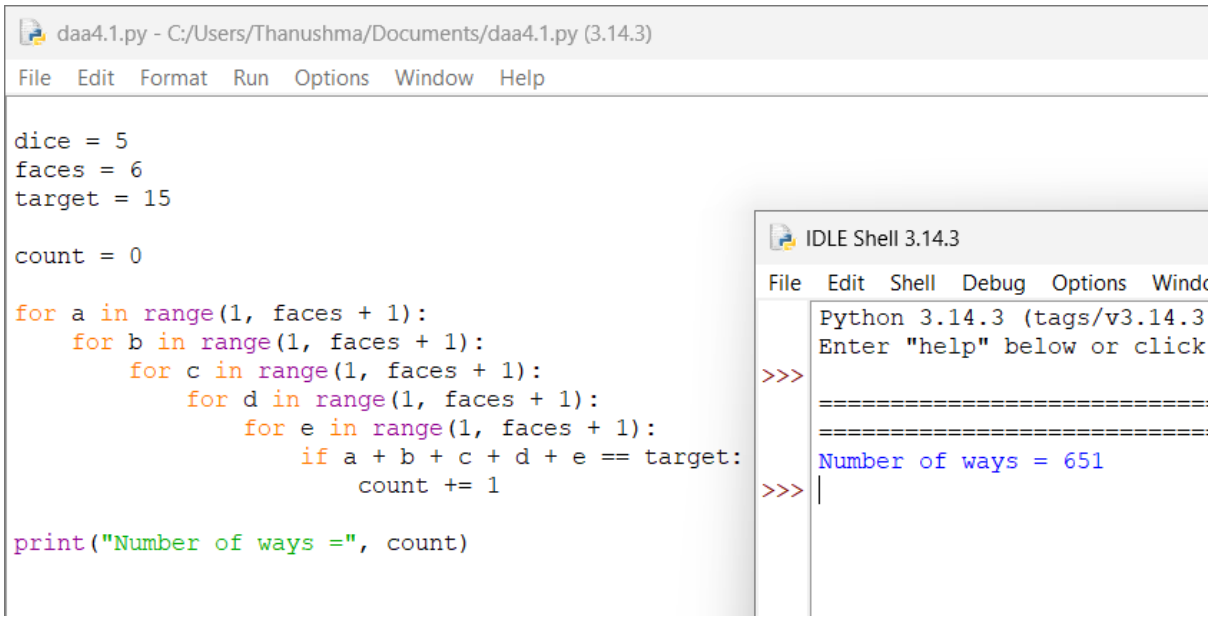
5. Quiz Competition Scoring

Aim: To write a Python program to find the number of valid scoring combinations when 5 six-faced dice have a total score of 15.

Algorithm

1. Start.
2. Set dice = 5, faces = 6, and target = 15.
3. Initialize count = 0.
4. Generate all possible dice outcomes.
5. Check whether their sum is equal to 15.
6. If yes, increment count.
7. Display the number of ways.
8. Stop

Execution:



Result: The program successfully finds 651 valid scoring combinations.

6.Sports Prediction System

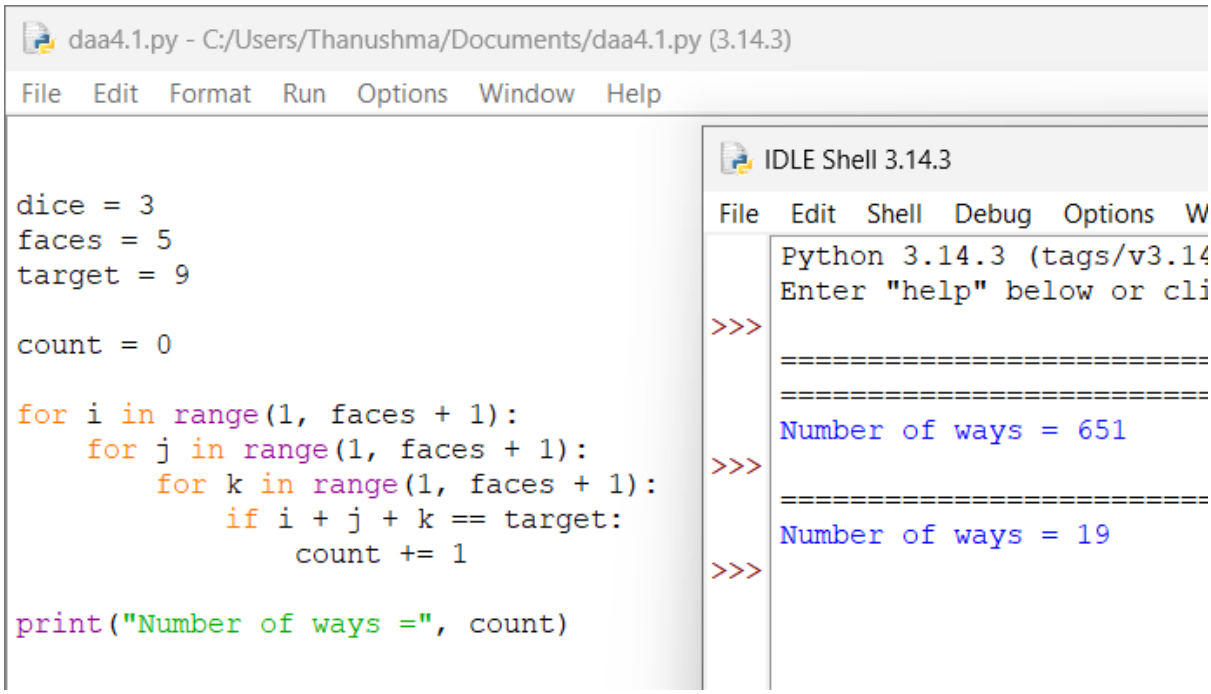
Aim: To write a Python program to determine the number of ways to obtain a predicted score of 9 using 3 five-faced dice.

Algorithm

1. Start.

2. Set dice = 3, faces = 5, and target = 9.
3. Initialize count = 0.
4. Generate all possible values of the three dice.
5. Check whether their sum is 9.
6. If yes, increment count.
7. Display the number of ways.
8. Stop.

Execution:



Result: The program successfully finds 19 possible outcomes.

7. Lottery Simulation System

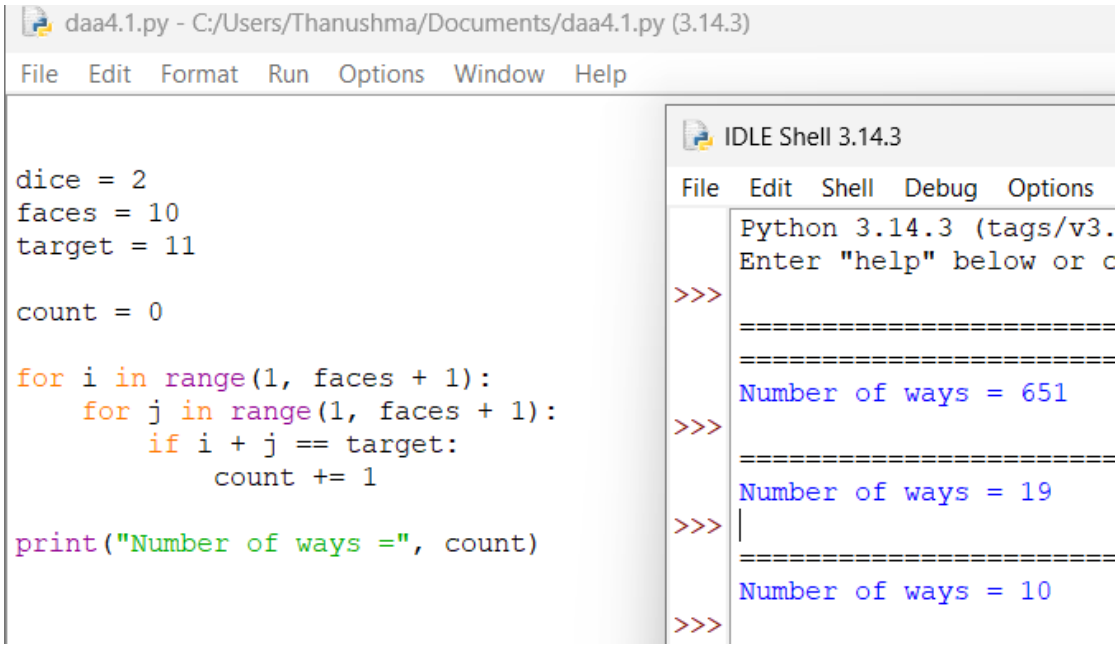
Aim: To write a Python program to find the number of possible combinations that result in a sum of 11 using 2 ten-faced dice.

Algorithm

1. Start.
2. Set dice = 2, faces = 10, and target = 11.
3. Initialize count = 0.

4. Generate all possible values for both dice.
5. Check whether their sum is 11.
6. If yes, increment count.
7. Display the number of ways.
8. Stop.

Execution:



Result: The program successfully finds 10 possible combinations**.**

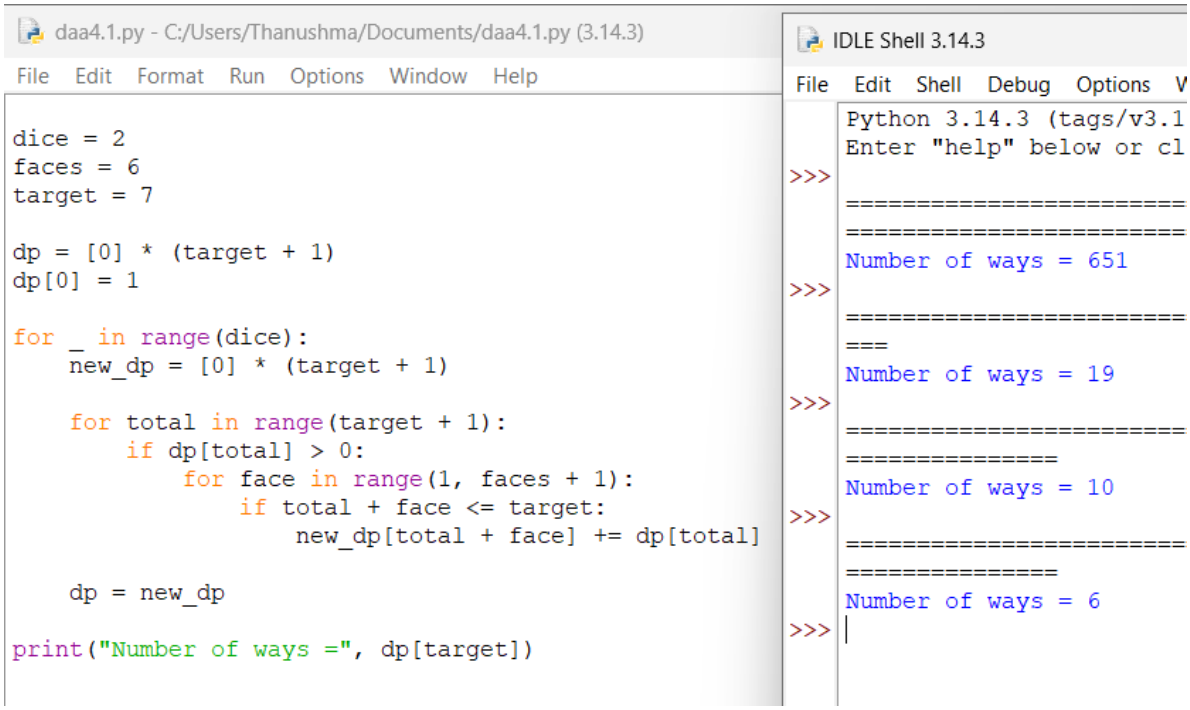
8.N Dice and M Faces

Aim: To write a Python program to find the number of ways to obtain a given sum using N dice, each having M faces.

Algorithm

1. Start.
2. Read the number of dice, faces, and target sum.
3. Use dynamic programming to calculate possible sums.
4. Store the number of ways for each sum.
5. Display the number of ways for the target sum.
6. Stop.

Execution:



Result: The program successfully calculates 6 ways to obtain the target sum

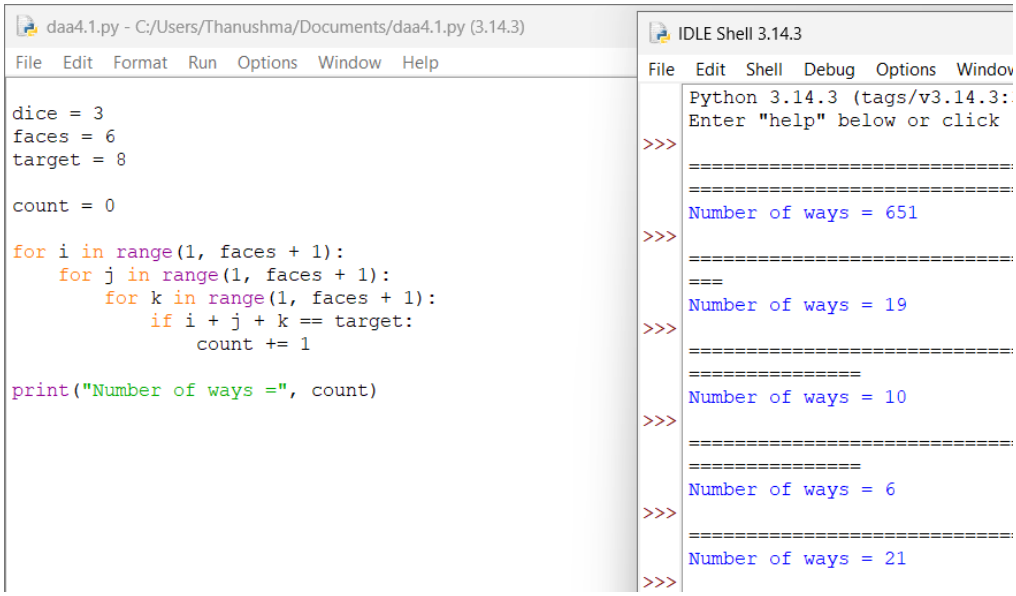
9.Target Score Using Multiple Dice

Aim: To write a Python program to determine the number of ways to obtain a target sum using 3 six-faced dice.

Algorithm

1. Start.
2. Set dice = 3, faces = 6, and target = 8.
3. Initialize count = 0.
4. Generate all possible outcomes.
5. Check whether the sum is 8.
6. Increment count when the condition is satisfied.
7. Display the result.
8. Stop.

Execution:



Result: The program successfully finds 21 possible ways**.**

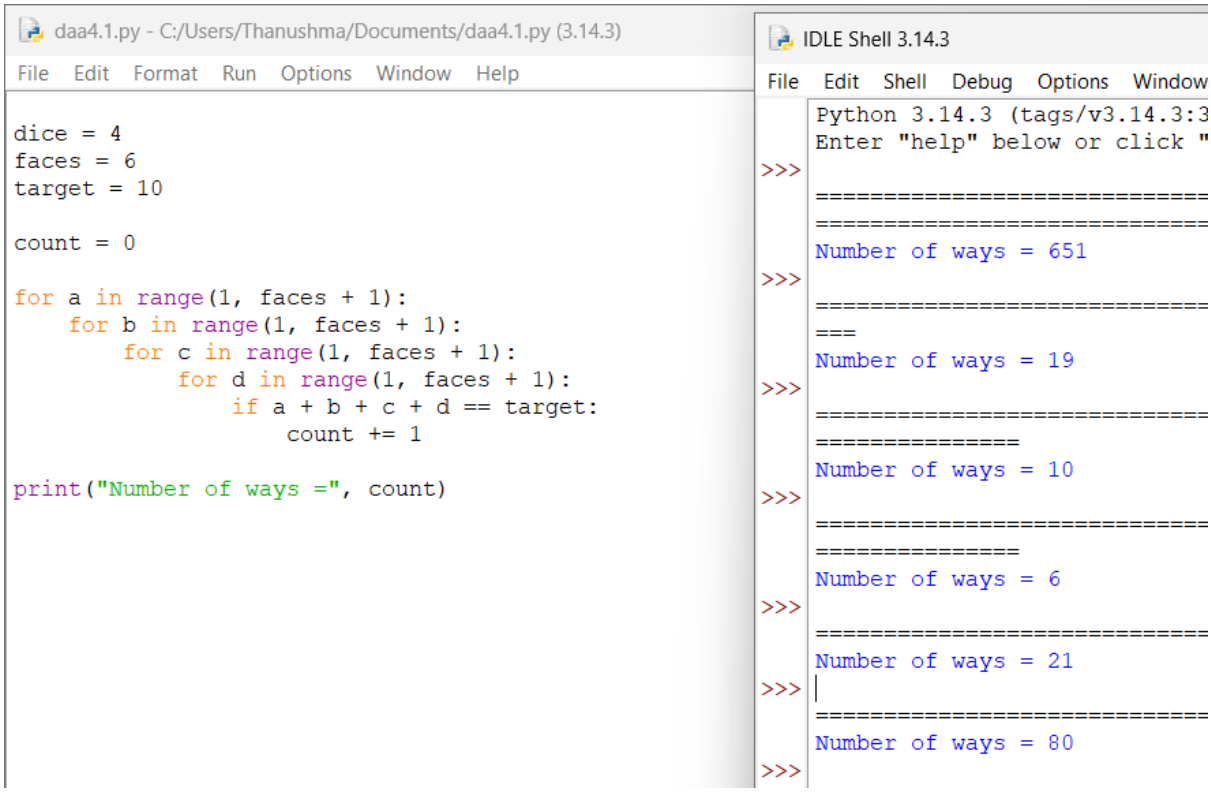
10.Specified Sum Outcomes

AimTo write a Python program to count the possible outcomes that produce a sum of 10 using 4 six-faced dice.

Algorithm

1. Start.
2. Set dice = 4, faces = 6, and target = 10.
3. Initialize count = 0.
4. Generate all possible values of four dice.
5. Check whether their sum is 10.
6. Increment count if the condition is satisfied.
7. Display the number of ways.
8. Stop.

Execution:



Result: The program successfully finds 80 possible outcome.

11.Required Score Dice Combinations

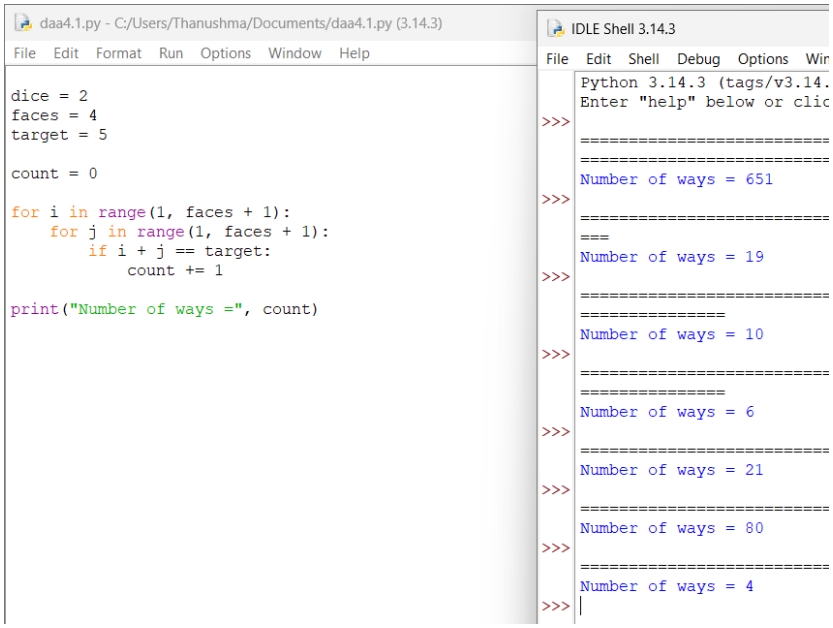
Aim: To write a Python program to find the number of possible dice combinations that yield a score of 5 using 2 four-faced dice.

Algorithm

1. Start.
2. Set dice = 2, faces = 4, and target = 5.
3. Initialize count = 0.
4. Generate all possible values for both dice.
5. Check whether their sum is 5.
6. Increment count if the sum is 5.
7. Display the result.

8. Stop.

Execution:



Result: The program successfully finds 4 possible combinations.

Binomial Coefficient

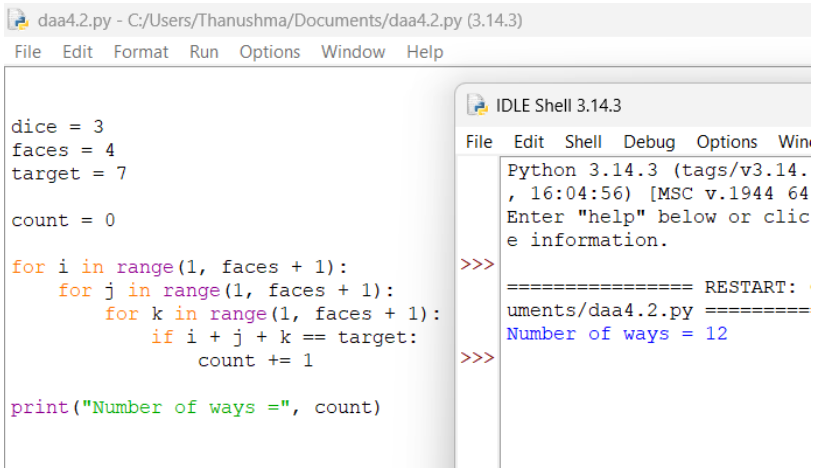
1. Dice Throw Target Sum

Aim: To write a Python program to determine the number of ways to obtain a target sum using dice throws.

Algorithm

1. Start.
2. Set the number of dice, faces, and target sum.
3. Generate all possible dice outcomes.
4. Check whether their sum equals the target.
5. Count the valid outcomes.
6. Display the number of ways.
7. Stop.

Execution:



Result: Thus, there are 12 ways to obtain the target sum of 7.

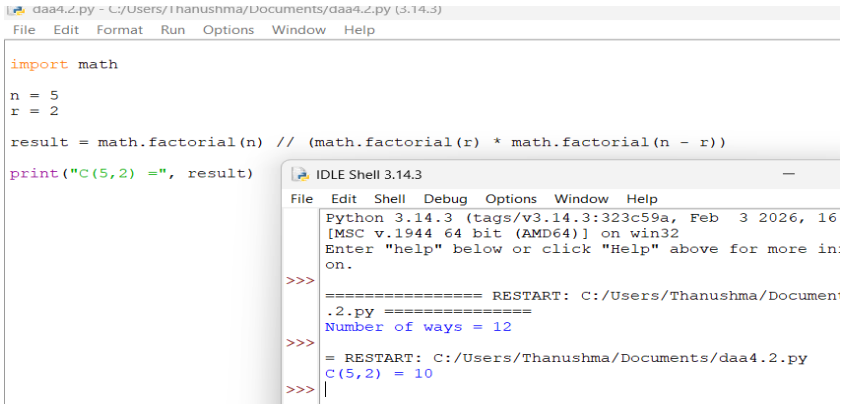
2.Binomial Coefficient

Aim: To write a Python program to calculate the Binomial Coefficient C(n,r).

Algorithm

1. Start.
2. Read n and r.
3. Calculate n!, r!, and (n-r)!.
4. Use the formula C(n,r) = n! / (r!(n-r)!).
5. Display the result.
6. Stop.

Execution:



Result: Thus, C(5,2) = 10.

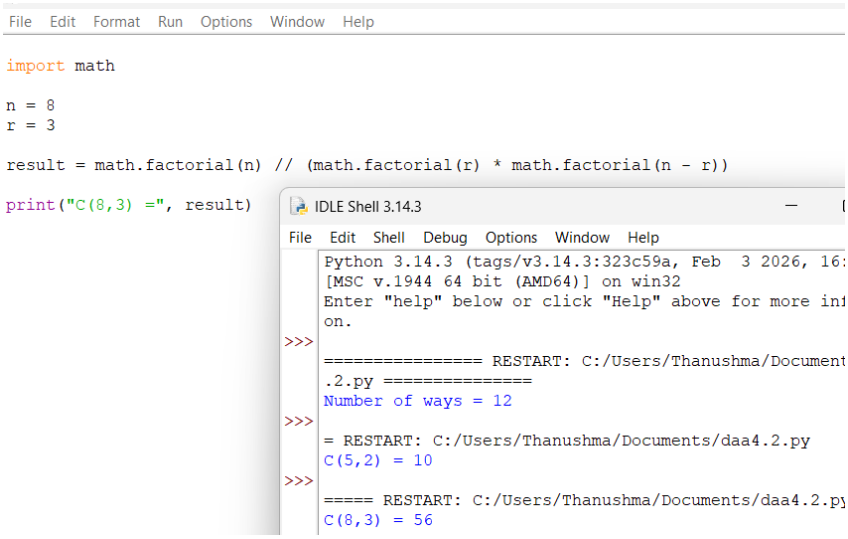
3.Selection of Items

Aim: To write a Python program to find the number of ways to select r items from n items.

Algorithm

1. Start.
2. Set n = 8 and r = 3.
3. Calculate the binomial coefficient.
4. Display the result.
5. Stop.

Execution:



Result: Thus, there are 56 ways to select 3 items from 8 items.

4.Binomial Coefficient Using Dynamic Programming

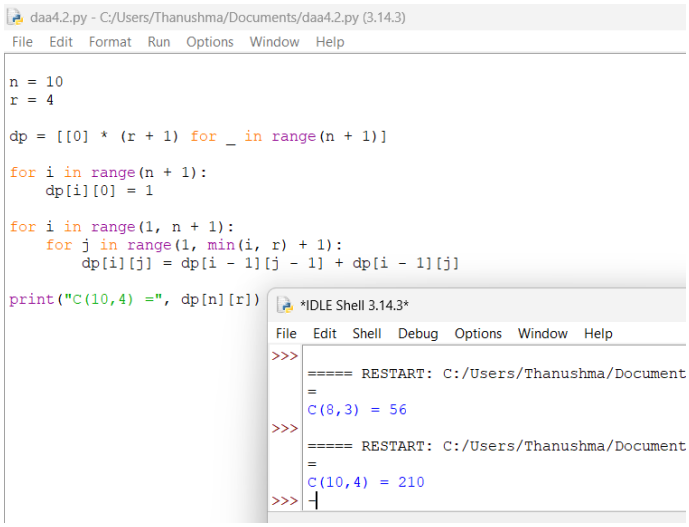
Aim: To write a Python program to compute the Binomial Coefficient using Dynamic Programming.

Algorithm

1. Start.

2. Create a DP table.
3. Set the boundary values to 1.
4. Calculate each value using Pascal's identity.
5. Obtain C(n,r) from the table.
6. Display the result.
7. Stop.

Execution:



Result: Thus, C(10,4) = 210.

5. Committee Formation

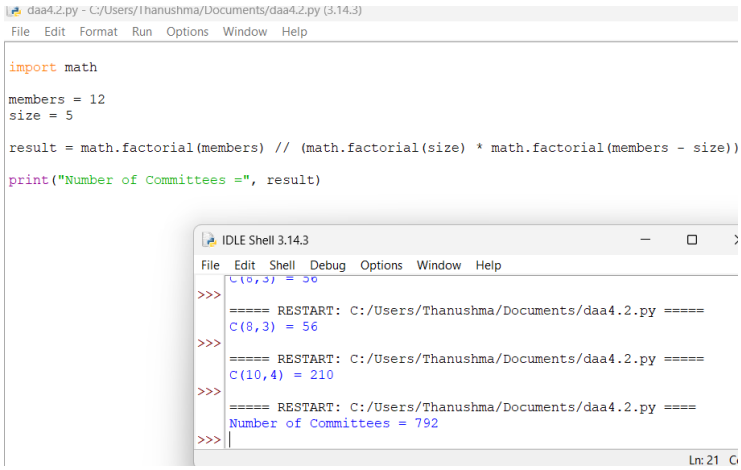
Aim: To write a Python program to determine the number of possible committees formed from a group of members.

Algorithm

1. Start.
2. Set total members as 12 and committee size as 5.
3. Calculate C(12,5).
4. Display the number of committees.

5. Stop.

Execution:



Result: Thus, 792 committees can be formed.

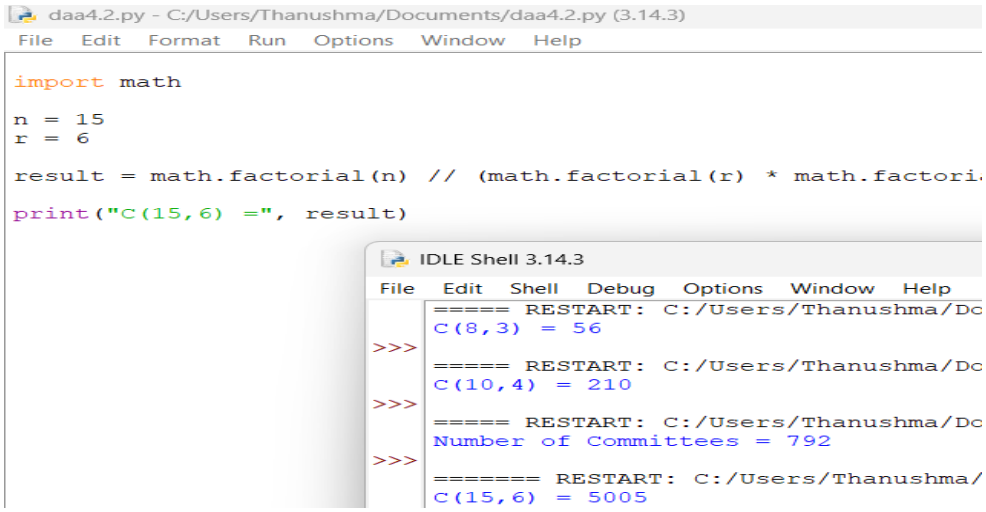
6. Combination Value

Aim: To write a Python program to calculate the combination value for given n and r.

Algorithm

1. Start.
2. Set n = 15 and r = 6.
3. Calculate the binomial coefficient.
4. Display the result.
5. Stop.

Execution:



Result: Thus, C(15,6) = 5005.

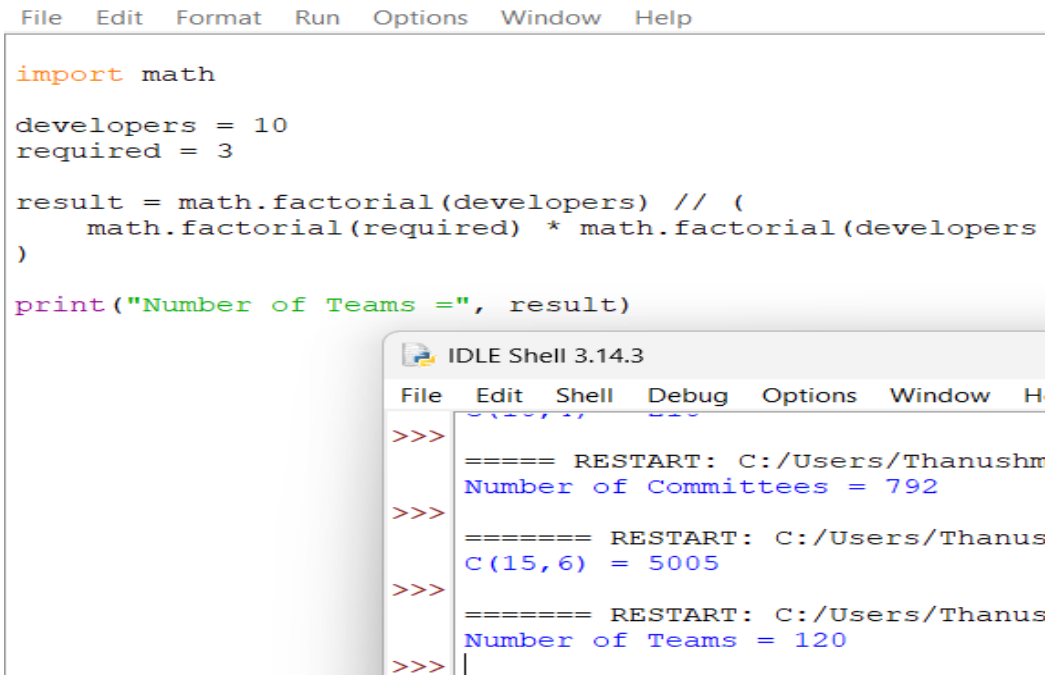
7.AI Project Team Selection

Aim: To write a Python program to calculate the number of different teams that can be formed by selecting 3 developers from 10 developers**.**

Algorithm

1. Start.
2. Set total developers as 10 and required developers as 3.
3. Calculate C(10,3).
4. Display the number of teams.
5. Stop.

Execution:



Result: Thus, 120 different teams can be formed.

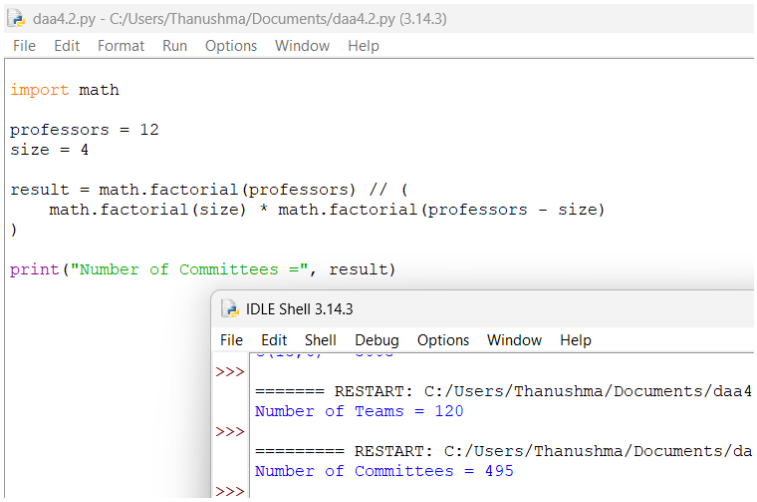
8.Scholarship Committee Selection

Aim: To write a Python program to determine the number of possible committees by selecting 4 professors from 12 professors.

Algorithm

1. Start.
2. Set total professors as 12 and committee size as 4.
3. Calculate C(12,4).
4. Display the number of committees.
5. Stop.

Execution:



Result: Thus, 495 possible committees can be formed.

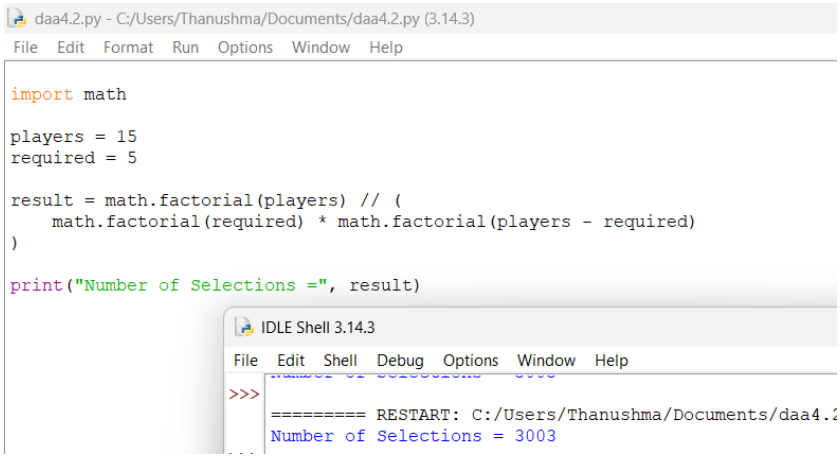
9.Cricket Tournament Squad

Aim: To write a Python program to calculate the number of possible selections of 5 players from 15 players.

Algorithm

1. Start.
2. Set total players as 15 and required players as 5.
3. Calculate C(15,5).
4. Display the number of selections.
5. Stop.

Execution:



Result: Thus, 3003 possible selections can be made.

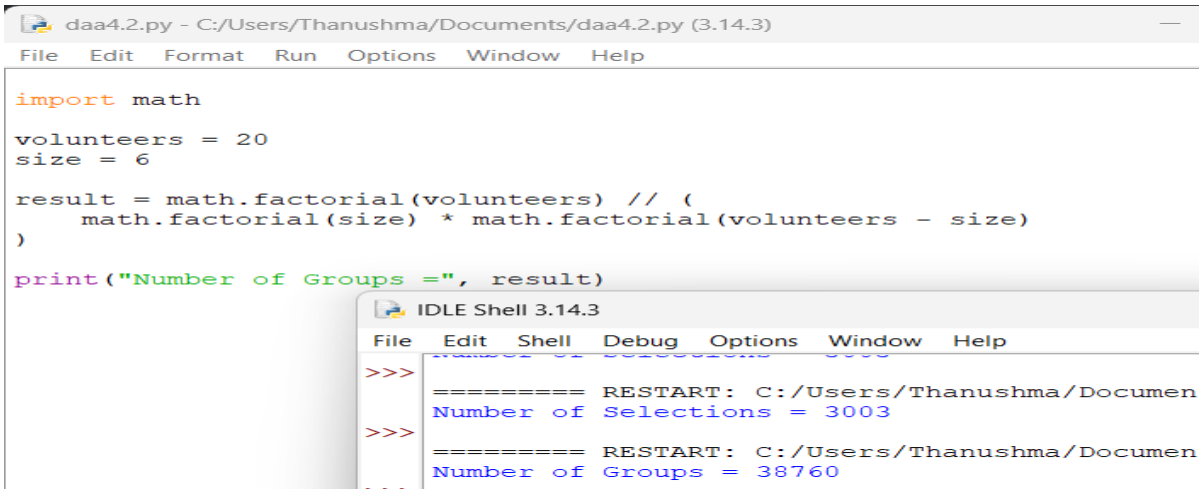
10.Product Testing Groups

Aim: To write a Python program to determine the number of groups formed by selecting 6 customers from 20 volunteers.

Algorithm

1. Start.
2. Set total volunteers as 20 and group size as 6.
3. Calculate C(20,6).
4. Display the number of groups.
5. Stop.

Execution:



Result: Thus, 38,760 testing groups can be formed.

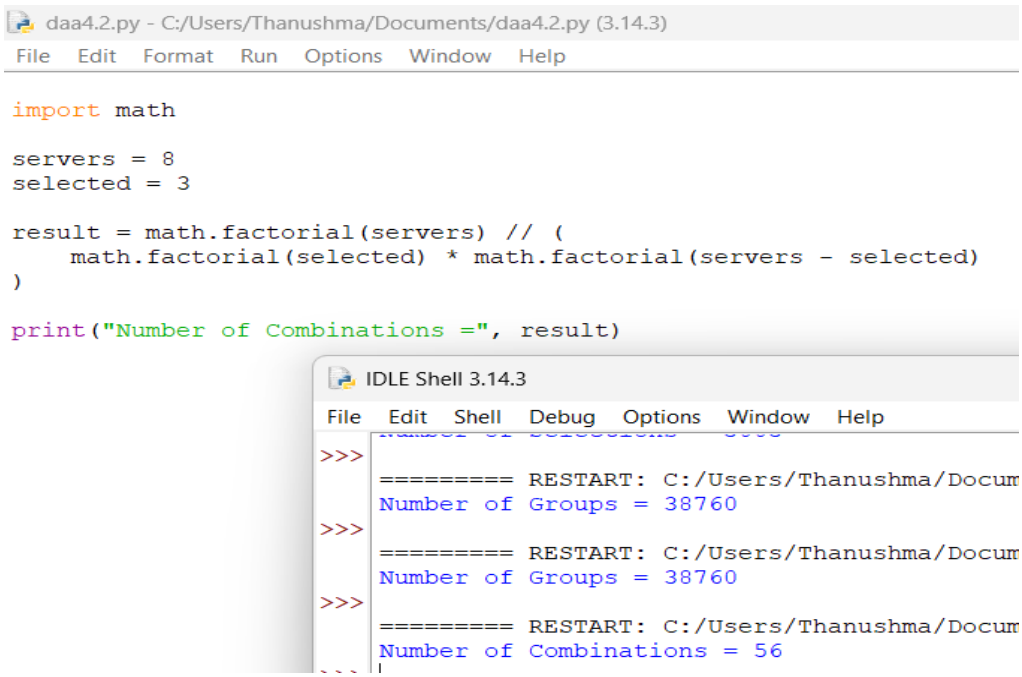
11. Server Combinations

Aim: To write a Python program to determine the number of possible combinations by selecting 3 servers from 8 servers.

Algorithm

1. Start.
2. Set total servers as 8 and selected servers as 3.
3. Calculate C(8,3).
4. Display the number of combinations.
5. Stop.

Execution:



Result: Thus, 56 possible server combinations can be formed.

Assembly Line Scheduling

1. Minimum Time Required to Manufacture a Product Using Two Assembly Lines

Aim: To write a program using Dynamic Programming to find the minimum time required to manufacture a product through two assembly lines, considering entry time, exit time, processing time, and transfer time.

Algorithm

1. Initialize the entry times, exit times, station processing times, and transfer times.
2. Calculate the minimum time to reach the first station of each line:
    - T1 = Entry1 + Line1[0]
    - T2 = Entry2 + Line2[0]
3. For every remaining station:
    - Calculate the minimum time to reach Line 1.
    - Calculate the minimum time to reach Line 2.
4. Add the corresponding exit time to both lines.
5. The smaller of the two final times is the minimum production time.

Execution:

```
entry = [10, 12]
exit = [18, 7]

line1 = [4, 5, 3, 2]
line2 = [2, 10, 1, 4]

transfer1 = [7, 4, 5]
transfer2 = [9, 2, 8]

t1 = entry[0] + line1[0]
t2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_t1 = min(t1 + line1[i],
                 t2 + transfer2[i - 1] + line1[i])

    new_t2 = min(t2 + line2[i],
                 t1 + transfer1[i - 1] + line2[i])

    t1 = new_t1
    t2 = new_t2

t1 += exit[0]
t2 += exit[1]

minimum_time = min(t1, t2)

print("Minimum Production Time =", minimum_time)
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a
, Feb  3 2026, 16:04:56) [MSC v.194
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help"
above for more information.
>>>
========== RESTART: C:/Use
rs/Thanushma/Desktop/1.PY ==========
==========
Minimum Production Time = 35
>>>
```

Result: Minimum Production Time = 35

2. Optimal Assembly Line Path with Minimum Processing Time

Aim: To write a program to determine the optimal assembly line path that gives the minimum processing time using dynamic programming.

Algorithm

1. Initialize entry times, exit times, station processing times, and transfer times.
2. Calculate the initial time for both assembly lines.
3. For each station, calculate:
    - Time if continuing on the same line.
    - Time if transferring from the other line.
4. Select the smaller time for each station.
5. Add the exit time of each line.
6. Select the smaller final value as the minimum production time.
7. The selected transitions represent the optimal assembly path.

Execution:

File Edit Format Run Options Window Help

```
entry = [8, 10]
exit = [12, 15]

line1 = [5, 6, 4]
line2 = [7, 3, 5]

transfer1 = [2, 1]
transfer2 = [3, 2]

t1 = entry[0] + line1[0]
t2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_t1 = min(t1 + line1[i],
                 t2 + transfer2[i - 1] + line1[i])

    new_t2 = min(t2 + line2[i],
                 t1 + transfer1[i - 1] + line2[i])

    t1 = new_t1
    t2 = new_t2

t1 += exit[0]
t2 += exit[1]

minimum_time = min(t1, t2)

print("Minimum Production Time =", minimum_time)
```

```
========== RESTART: C:/Users/Thanushma/Documents/2.py ==========
Minimum Production Time = 35
```

Result: Minimum Production Time = 35

3. Fastest Route Through Assembly Stations

Aim: To write a program to compute the fastest route through assembly stations on two assembly lines by minimizing the total processing and transfer time.

Algorithm

1. Read the entry and exit times of the two assembly lines.
2. Read the processing time for each station.
3. Read the transfer times between the two lines.
4. Initialize the time required to reach the first station.
5. For every next station:
    - Calculate the time for staying on the same line.
    - Calculate the time for transferring from the other line.
6. Choose the minimum of these two times.
7. Add the exit time after the final station.
8. Compare the final times of both lines.
9. Display the smaller value as the fastest production time.

Execution:

```
entry = [5, 6]
exit = [4, 5]

line1 = [3, 4, 5]
line2 = [4, 3, 6]

transfer1 = [2, 1]
transfer2 = [1, 2]

t1 = entry[0] + line1[0]
t2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_t1 = min(t1 + line1[i],
                 t2 + transfer2[i - 1] + line1[i])

    new_t2 = min(t2 + line2[i],
                 t1 + transfer1[i - 1] + line2[i])

    t1 = new_t1
    t2 = new_t2

t1 += exit[0]
t2 += exit[1]

minimum_time = min(t1, t2)

print("Minimum Production Time =", minimum_time)
```

```
=== RESTART: C:/Users/Thanushma/Documents/2.py ==
Minimum Production Time = 21
```

Result: Minimum Production Time = 21

4. Assembly Line Scheduling

Aim: To write a Python program to identify the assembly line schedule that minimizes the total manufacturing time.

Algorithm

1. Start.
2. Initialize entry times, exit times, processing times, and transfer times.
3. Calculate the minimum time to reach each station on Line 1 and Line 2.
4. At each station, compare staying on the same line with switching from the other line.
5. Add the corresponding exit time after the final station.
6. Select the minimum of the two final times.
7. Display the minimum production time.
8. Stop.

Execution:

File Edit Format Run Options Window Help

```
entry = [7, 9]
exit = [8, 6]

line1 = [4, 3, 5, 2]
line2 = [5, 2, 4, 3]

transfer1 = [1, 2, 1]
transfer2 = [2, 1, 2]

n = len(line1)

time1 = entry[0] + line1[0]
time2 = entry[1] + line2[0]

for i in range(1, n):
    new_time1 = min(
        time1 + line1[i],
        time2 + transfer2[i - 1] + line1[i]
    )

    new_time2 = min(
        time2 + line2[i],
        time1 + transfer1[i - 1] + line2[i]
    )

    time1 = new_time1
    time2 = new_time2

result = min(time1 + exit[0], time2 + exit[1])

print("Minimum Production Time =", result)
```

```
>>
=== RESTART: C:/Users/Thanushma/Documents/2.py ==
Minimum Production Time = 27
>>
```

5.Minimum Cost Path Through Assembly System

Aim: To write a Python program to determine the minimum production time through a two-line assembly system.

Algorithm

1. Start.
2. Initialize entry, exit, processing, and transfer times.
3. Calculate the initial time for both assembly lines.
4. For each station, calculate the minimum time by either staying on the current line or switching lines.
5. Add the exit time.
6. Select the smaller final value.
7. Display the minimum production time.
8. Stop.

Execution:

```
entry = [6, 7]
exit = [5, 4]

line1 = [2, 4, 3]
line2 = [3, 2, 5]

transfer1 = [1, 2]
transfer2 = [2, 1]

time1 = entry[0] + line1[0]
time2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_time1 = min(
        time1 + line1[i],
        time2 + transfer2[i - 1] + line1[i]
    )

    new_time2 = min(
        time2 + line2[i],
        time1 + transfer1[i - 1] + line2[i]
    )

    time1 = new_time1
    time2 = new_time2

result = min(time1 + exit[0], time2 + exit[1])

print("Minimum Production Time =", result)
```

```
=== RESTART: C:/Users/Thanushma/Documents/2.py =
Minimum Production Time = 20
>
```

Result: Minimum Production Time = 20

6.Automobile Manufacturing Assembly Lines

Aim: To write a Python program to determine the minimum time required to manufacture a car using two assembly lines.

Algorithm

1. Start.
2. Initialize entry, exit, station processing, and transfer times.
3. Calculate the initial processing time on both lines.
4. For each subsequent station, calculate the minimum time for both lines.
5. Consider both staying on the same line and transferring from the other line.
6. Add the appropriate exit time.
7. Select the minimum final time.
8. Stop.

Execution:

```
entry = [10, 12]
exit = [18, 7]

line1 = [4, 5, 3, 2]
line2 = [2, 10, 1, 4]

transfer1 = [7, 4, 5]
transfer2 = [9, 2, 8]

time1 = entry[0] + line1[0]
time2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_time1 = min(
        time1 + line1[i],
        time2 + transfer2[i - 1] + line1[i]
    )

    new_time2 = min(
        time2 + line2[i],
        time1 + transfer1[i - 1] + line2[i]
    )

    time1 = new_time1
    time2 = new_time2

result = min(time1 + exit[0], time2 + exit[1])

print("Minimum Production Time =", result)
```

```
=== RESTART: C:/Users/Thanushma/Documents/2.py ==
Minimum Production Time = 35
```

7.Smartphone Production Line

Aim: To write a Python program to determine the optimal route through two production lines for minimizing total smartphone assembly time.

Algorithm

1. Start.
2. Initialize entry, exit, processing, and transfer times.
3. Calculate the initial time on each production line.
4. At every station, calculate the minimum time for each line.
5. Consider both direct processing and line switching.
6. Add the final exit time.
7. Display the minimum production time.
8. Stop.

Execution:

```
entry = [8, 10]
exit = [12, 15]

line1 = [5, 6, 4]
line2 = [7, 3, 5]

transfer1 = [2, 1]
transfer2 = [3, 2]

time1 = entry[0] + line1[0]
time2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_time1 = min(
        time1 + line1[i],
        time2 + transfer2[i - 1] + line1[i]
    )

    new_time2 = min(
        time2 + line2[i],
        time1 + transfer1[i - 1] + line2[i]
    )

    time1 = new_time1
    time2 = new_time2

result = min(time1 + exit[0], time2 + exit[1])

print("Minimum Production Time =", result)
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59
3 2026, 16:04:56) [MSC v.1944 64 b
4)] on win32
Enter "help" below or click "Help"
or more information.
>>>
========== RESTA
sers/Thanushma/Documents/3.py =====
==========
Minimum Production Time = 35
>>>
```

Result: The minimum production time calculated from the given input is 35.

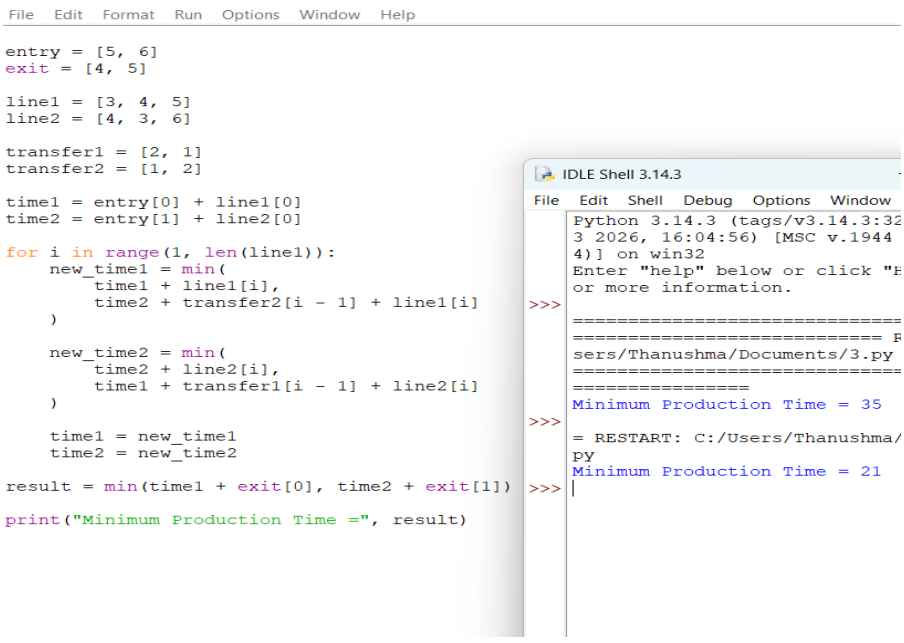
8.Food Packaging Assembly Lines

Aim: To write a Python program to find the minimum completion time for products passing through two packaging lines.

Algorithm

1. Start.
2. Initialize entry, exit, processing, and transfer times.
3. Calculate the initial time for both lines.
4. For each station, calculate the minimum time considering both possible lines.
5. Add the exit time after the final station.
6. Select the minimum final time.
7. Display the result.
8. Stop.

Execution:



Result: The minimum completion time is 21 units of time**.**

9. Medicine Bottle Production

Aim: To write a Python program to determine the optimal production schedule for medicine bottles using two assembly lines.

Algorithm

1. Start.
2. Initialize all assembly line parameters.
3. Calculate the initial time for both lines.
4. For each station, compare staying on the current line and transferring from the other line.
5. Update the minimum time.
6. Add the appropriate exit time.
7. Display the minimum production time.
8. Stop.

Execution:

```
entry = [7, 9]
exit = [8, 6]

line1 = [4, 3, 5, 2]
line2 = [5, 2, 4, 3]

transfer1 = [1, 2, 1]
transfer2 = [2, 1, 2]

time1 = entry[0] + line1[0]
time2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_time1 = min(
        time1 + line1[i],
        time2 + transfer2[i - 1] + line1[i]
    )

    new_time2 = min(
        time2 + line2[i],
        time1 + transfer1[i - 1] + line2[i]
    )

    time1 = new_time1
    time2 = new_time2

result = min(time1 + exit[0], time2 + exit[1])

print("Minimum Production Time =", result)
```

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb
3 2026, 16:04:56) [MSC v.1944 64 bit (AMI
4)] on win32
Enter "help" below or click "Help" above
or more information.
>>>
========== RESTART: C:/
sers/Thanushma/Documents/3.py ==========
==========
Minimum Production Time = 35
>>>
= RESTART: C:/Users/Thanushma/Documents/
py
Minimum Production Time = 21
>>>
= RESTART: C:/Users/Thanushma/Documents/
py
Minimum Production Time = 27
>>>
```

Result: The minimum production time calculated from the given input is 27.

10. Circuit Board Manufacturing

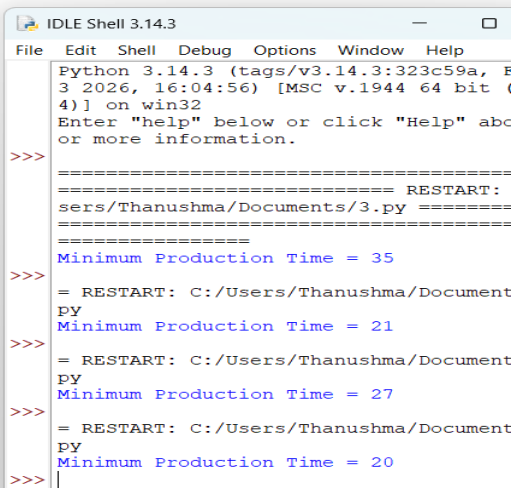
Aim: To write a Python program to find the minimum manufacturing time in a two-line circuit board assembly system.

Algorithm

1. Start.
2. Initialize entry, exit, processing, and transfer times.
3. Calculate the initial time on both lines.
4. For every remaining station, calculate the minimum possible time.
5. Consider both staying and switching lines.
6. Add the exit time.
7. Select the minimum final time.
8. Stop.

Execution:

```
entry = [6, 7]
exit = [5, 4]

line1 = [2, 4, 3]
line2 = [3, 2, 5]

transfer1 = [1, 2]
transfer2 = [2, 1]

time1 = entry[0] + line1[0]
time2 = entry[1] + line2[0]

for i in range(1, len(line1)):
    new_time1 = min(
        time1 + line1[i],
        time2 + transfer2[i - 1] + line1[i]
    )

    new_time2 = min(
        time2 + line2[i],
        time1 + transfer1[i - 1] + line2[i]
    )

    time1 = new_time1
    time2 = new_time2

result = min(time1 + exit[0], time2 + exit[1])

print("Minimum Production Time =", result)
```



Result: The minimum manufacturing time calculated from the given input is 20 units of time.